Why Linked Lists Are (Almost Always) a Bad Idea in HPC

Course: High Performance Computing 1 (DSAI + SDIC)
Lecturer: Luca Tornatore
Academic Year: 2025/2026

---

1. The setup

This note has been moved from an interesting discussion during a lecture, on a point which appears simple but turns out to have several layers of subtlety: why are linked lists, one of the first data structures you learn in any computer science course, so catastrophically bad for performance on modern hardware?

The answer is not a single reason. It is a cascade of interacting mechanisms, each of which would be bad enough on its own, and which together make the linked list one of the worst possible choices for any performance-critical traversal. We will go through them one by one, starting from the least important and working our way up to the fundamental architectural limitation that, by itself, would condemn the data structure even if all the other problems were somehow solved.
This topic offers a nice opportunity to put together so many - virtually all - topics about performance due to memory and prefetching.

Remind: the context throughout is HPC — large-scale numerical computation, the kind of work where you have millions/billions of elements and you traverse them in a tight loop, over and over. In *this* context, saying "linked lists are bad" is not an opinion; it is a quantitative statement about hardware physics.

In the following I'm discussing the cons in increasing order of importance/impact (so to comply with the saying "last but not least"...)

---

2. The overhead tax

The first and least significant issue is memory overhead. Every node in a linked list carries a pointer to the next node — 8 bytes on a 64-bit system. If the list is doubly linked, that is 16 bytes. This is pure structural overhead: it carries no useful data.

But the pointer is not the full story. Each node is typically allocated with `malloc()`, and `malloc()` has its own bookkeeping, as we have seen. On glibc's allocator (ptmalloc2, which is what you get on most Linux systems), every allocated chunk carries a header: at minimum a `size` field and a `prev_size` field, for a total of 16 bytes of allocator metadata. Furthermore, glibc enforces a minimum chunk size of 32 bytes (on 64-bit), so even `malloc(1)` consumes 32 bytes. And all chunks are aligned to 16-byte boundaries, which may introduce additional padding.

The practical consequence: for a node containing a single `double` (8 bytes) and a `next` pointer (8 bytes), the total allocation is typically 48 bytes — 16 of useful data and 32 of overhead and padding. That is a 67% space waste. For a node containing 100 doubles and a pointer, the useful data is 808 bytes and the overhead is maybe 16–32 bytes — a 2–4% waste, which is perfectly tolerable.

So the overhead tax is size-dependent. For large nodes it is negligible; for small nodes it is absurd. But even in the worst case, this is a *capacity* problem, not a *latency* problem. It means your data takes more space in cache and in memory, but it does not, by itself, explain why the traversal is slow. The reasons for that are deeper.

---

3. The construction cost

Every node in a linked list is individually allocated with `malloc()`. The cost of a single `malloc()` call on a modern glibc is not catastrophic — for small allocations it typically amounts to a free-list lookup, some pointer manipulation, and possibly a lock acquisition if multiple threads share an arena. On the order of 50–200 ns per call, amortised.

Plus, this cost is paid during *construction*, not during traversal. In an HPC code, you typically build the data structure once (or at most a few times) and then traverse it millions of times in the hot loop. The construction cost, even if it amounts to $N \times 100$ ns for N nodes, is amortised over the lifetime of the computation and is almost never the bottleneck.

The real legacy of `malloc()`-per-node is *indirect*: it is the allocation pattern that produces the memory fragmentation discussed in the next section. The `malloc()` call itself is not the problem; what `malloc()` does to your memory layout is.

There is, however, one scenario where construction cost does matter: if you frequently insert and delete nodes (as in a dynamic simulation with particles entering and leaving the domain). In that case the `malloc()`/`free()` calls are interleaved with computation, they fragment the heap over time, and they can trigger expensive operations like `mmap()`/`munmap()` when the allocator needs to grow or shrink arenas.

In such a scenario, the solution is usually a *pool allocator*: pre-allocate a large block and hand out nodes from it, bypassing `malloc()` entirely. This eliminates the construction cost *and* the fragmentation problem in one stroke. It does not, however, fix the problems that follow.

---

4. Fragmentation and the destruction of spatial locality

Here is where the real damage begins.

When you allocate an array of N elements, you get a single contiguous block of memory. Iterating over it is a sequential scan: element k is at address `base + k × sizeof(element)`, which increases monotonically. The **hardware prefetcher** (keyword:: we'll see it later) sees a simple stride pattern, the cache lines are fully consumed (every byte is useful data), and the access is as cache-friendly as physically possible.

When you allocate N linked-list nodes with `malloc()`, you get N *separate* allocations. Where each one lands in the virtual address space depends on the allocator's internal state: which arenas have free space, what sizes are available, how much fragmentation has accumulated from previous allocations and frees.

Now, to be honest, there is an important nuance here. If you allocate all N nodes in a tight loop, with no intervening `free()` calls or other `malloc()` calls, most allocators will hand out the nodes from a contiguous region of the heap, one after the other, in roughly sequential order. In this case, the spatial locality is actually not terrible — not as good as an array (because of the per-node allocator headers interleaved with the data), but not catastrophic either. You will see this in your experiments with provided codes if you are not careful: a freshly-allocated linked list can appear surprisingly well-behaved in terms of cache misses.

The fragmentation becomes a disaster in two scenarios:

Scenario A: interleaved allocations. If you allocate linked-list nodes for *multiple* data structures concurrently, or if you allocate nodes interleaved with other objects (matrices, buffers, temporary arrays), the nodes of any one list end up scattered among unrelated data. The spatial locality is destroyed not because `malloc()` is broken, but because you are asking it to interleave allocations from different sources.

Scenario B: dynamic insertion and deletion. After a sequence of insertions and deletions, the free-list inside the allocator becomes fragmented. New allocations may be served from freed chunks that are scattered across the heap. Even if the list was originally contiguous, after enough churn it will be dispersed across disparate memory regions.

In both scenarios, *traversing the list means jumping unpredictably across memory*. Each node access may land on a different cache line; worse, a significant fraction of each loaded cache line is wasted because it contains data belonging to other objects or to the allocator's own metadata, not to the next node in the list.

If the nodes were contiguous, you would load one cache line every k elements (where $k =$LINESIZE / \text{sizeof(node)}$), and the prefetcher would handle most of the latency. When nodes are scattered, every single node access is a potential cache miss, and every miss pays the full penalty.

For an N-body simulation with, say, 10^7 particles, where each particle is a node with coordinates, velocity, mass, and some auxiliary data — perhaps 128 bytes — the difference between a contiguous array and a scattered linked list is not a few percent. It can easily be an order of magnitude.

---

5. Pointer chasing and the collapse of memory-level parallelism

This is, in my view, the most important point; the one that would condemn linked lists with no doubt, even if all the previous problems were somehow solved.

Let's start with a key observation : in a linked-list traversal, the address of node $k+1$ is stored *inside* node k. You cannot even begin to fetch node $k+1$ from memory until you have loaded the `next` pointer from node k — and that load cannot complete until node k is in the cache.
The traversal is a *dependent load chain*: each memory access depends on the result of the previous one.

This has a devastating consequence for memory-level parallelism (MLP).

A modern out-of-order core can sustain many outstanding cache misses simultaneously. On Intel Skylake and its descendants, there are 12 Line Fill Buffers (LFBs) in the L1 data cache and up to 32 outstanding requests to L2. This means that when you iterate over an array, the hardware can look ahead, see that you will need cache lines $k+1, k+2, \ldots, k+10$, and issue all those requests concurrently. The total time to traverse N elements is roughly

$T_{\text{array}} \approx \frac{N \times L_{\text{miss}}}{P}$

where L_{miss} is the miss penalty and P is the degree of parallelism (the number of outstanding misses the hardware can sustain). With $P \approx 10$ and $L_{\text{miss}} = 200$ cycles (for a DRAM miss), each element effectively costs about 20 cycles.
(I did not mention that in the lecture about memory hierarchy because I was already piling up lot of details, but now it makes sense to motivate my *tranchant* statements on linked lists).

For a linked list, $P = 1$. You cannot issue the request for node $k+1$ until node k's `next` pointer has arrived from memory. The total time is

$$T_{\text{list}} \approx N \times L_{\text{miss}}$$

with no division by P. Every single node access pays the full latency, serially.

This is not a cache miss rate problem — even if you had a magical cache with 100% hit rate on the node data, you would still need to serialise the `next` pointer loads.
The issue is structural: the *information* needed to compute the next address is only available after the previous access completes. No amount of cache tuning, prefetching configuration, or memory layout optimisation can fix this.
It is a fundamental property of the data structure.
Very sad.

Can the hardware prefetcher help?

Sadly, no.
Hardware prefetchers (we'll see a bit of this in the section about prefetching in the slides) work by detecting *patterns* in the stream of memory addresses: constant strides, ascending/descending sequences, repeated access patterns.
A linked list that in principle may be fragmented across memory has no detectable pattern — the address of the next node is, from the prefetcher's perspective, essentially random. The prefetcher will not even attempt to predict it.
Actually this is a general problem of pointer-chasing, not just of linked lists

Even for a linked list whose nodes happen to be contiguous in memory (because they were allocated sequentially; so in any case there will be the gap of ~16B info block at the beginning), the hardware sees the program loading a `next` pointer, computing an address, and issuing a load to that address. The stride between consecutive loads is node-sized, which is regular, and a stride-based prefetcher *might* pick it up. But this is fragile: if even a few nodes are out of order (due to insertions, deletions, or reallocation), the stride pattern breaks and the prefetcher gives up.

Can software prefetching help?

Partially. If you have access to `node->next` (which you do, once `node` is in cache), you can issue a software prefetch (we'll see that in the prefetching section) for `node->next` while processing `node`:

```
while (node != NULL) {
    __builtin_prefetch(node->next, 0, 3);  // prefetch next node
    // ... process node ...
    node = node->next;
}
```

This hides the latency of *one* link — you are processing node k while the prefetch for node $k+1$ is in flight. But you cannot prefetch node $k+2$ because you do not know its address yet (it is stored in node $k+1$, which has not arrived). The prefetch distance is structurally limited to 1. If the processing per node is long enough to hide one cache miss (say, 200 cycles of computation per node), this is sufficient. If the processing is short (a comparison, an accumulation — a few cycles), the prefetch buys you almost nothing because the miss latency dwarfs the processing time.

For deep pipelines, there is a trick called "pointer prefetching" where you maintain a small lookahead buffer of pre-fetched node addresses, but it requires complex code restructuring and its benefit is limited by the lookahead depth.
The greatest advancements in technology came from laziness: we have beautiful hw prefetchers, and you really want to adopt a data structure fundamentally flawed, and to write by hand the pointer prefetchng chain?

Let me put a punchline

For an array of N doubles traversed sequentially:

- The prefetcher handles everything
- MLP is maximised ($P \approx 10$–20)
- Each element costs approximately $L_{\mathrm{L1}} + \frac{L_{\mathrm{miss}}}{P \times k}$ where k is elements per cache line
- With $L_{\mathrm{L1}} = 4$ cycles, $L_{\mathrm{miss}} = 200$ cycles, $P = 10$, $k = 8$: roughly **6.5 cycles per element**

For a linked list of N nodes, scattered in memory:

- Prefetcher is useless
- MLP = 1
- Each element costs approximately L_{miss}
- With $L_{\mathrm{miss}} = 200$ cycles: roughly **200 cycles per element**
- you need to write by hand a non trivial code to attempt some sw prefetch of the chain of next pointers

That is a factor of 30×. And we have not yet discussed vectorisation, which makes it worse.

---

6. The TLB amplifier

On top of the serialised memory access pattern, there is an additional penalty that becomes severe for large data structures: TLB (Remind: Translation Lookaside Buffer) misses.

We have seen taht every memory access requires a virtual-to-physical address translation. This translation is cached in the TLB (remind: small, fast hardware cache of page table entries). We did not inspect how the TLB is made, but let not dig into too much details: just remember that on a TLB miss, i.e. if the page table entries themselves are not in cache, a full page table walk is triggered, which costs 100–1000+ cycles.

For an array that occupies a contiguous region of memory, the TLB coverage is excellent. With 4 KiB pages, each TLB entry covers 4096 bytes — that is 512 doubles. (some figures: ona modern Intel's Skylake, the L1 DTLB alone covers 64 × 4 KiB = 256 KiB, and the L2 STLB covers 1536 × 4 KiB ≈ 6 MiB). A sequential scan touches pages in order, so the TLB replacement pattern is optimal. With 2 MiB huge pages (as you may find on HPC platforms, at least), a single TLB entry covers 262,144 doubles; the entire L1 DTLB covers over 4 GB.

For a scattered linked list, each node might be on a different page. If you have N nodes scattered across M pages (where M can approach N for small nodes), and M exceeds the TLB capacity, then *every node access* incurs a TLB miss in addition to the cache miss. The TLB miss penalty is on top of the cache miss penalty — it is the cost of finding out *where* in physical memory the data is, before you can even begin to fetch it.

The compound cost per node becomes:

$$C_{\mathrm{node}} \approx L_{\mathrm{TLB\ walk}} + L_{\mathrm{cache\ miss}}$$

which, in the worst case, can be 300–500 cycles per access. Serialised. One at a time. For 10^7 nodes, that is 3×10^9 to 5×10^9 cycles — several seconds on a 3 GHz core, just to *traverse* the list without doing any computation.

Ok, as we commented, I'm indulging into dramatization. it is very unlikely that you get this scenario, **if you allocate the nodes one after the other in a tight loop**. You "just" add some inefficiency due to additional TLB pressure.

So, to be fair, let me stress that TLB pressure is a possible *amplifier*, not the root cause.
However, as we have seen in previsou paragraph 6), even with perfect TLB coverage — say, all nodes within a single 1 GiB huge page — the pointer-chasing serialisation from the previous section still applies. You would eliminate the TLB walk penalty but not the serialised cache misses. The TLB makes a bad situation worse; the pointer chasing is what makes it bad in the first place.

---

7. The Tombstone: vectorisation and GPU

There is one more penalty that is often overlooked in discussions of linked lists, and it is a big one: **linked-list traversal is impossible to vectorise.**

With an array, the compiler (or the programmer, via intrinsics) can process multiple elements simultaneously using SIMD instructions. On a modern x86-64 core with AVX2, you can load 4 doubles per instruction; with AVX-512, 8 doubles. This is a factor of 4–8× in computational throughput for operations like summation, search, filtering, or elementwise computation.

A linked-list traversal is inherently scalar. Each iteration depends on the previous one (via the `next` pointer), so there is no way to process nodes $k, k+1, k+2, k+3$ simultaneously — you do not know where they are until you have chased k's pointer, then $k+1$'s, and so on. The SIMD lanes are useless.

This means that even for the *computational* part of the traversal (not the memory accesses), you are leaving a factor of 4–8× on the table. Combined with the MLP collapse from pointer chasing and the cache/TLB penalties, the total performance difference between a linked list and an array can easily exceed two orders of magnitude for a memory-bound loop on a modern core.

And the final death sentence: on GPU you can not really have linked list (in general: *pointer chasing*) working efficiently, it is an absurdity that would trash the 20k€ that have been spent for the hardware.

---

8. Ah no. Let's add nails to the coffin

What if you are in multi-threading?
Laughable. In the Advanced HPC course we'll see an example and you'll realize how a nightmare it is.
I have purposely chosen that example – a linked list to implement a heap – because ensuring threads synchronization is a nightmare.

---

9. A summary of the damage

Let us collect all the effects and estimate their individual contribution for a concrete scenario. Suppose we have $N = 10^6$ nodes, each containing a few doubles, scattered across memory. We compare traversal of this structure as a linked list versus as a contiguous array.

Effect	Array	Linked list	Ratio
Spatial locality	Sequential; cache lines fully used	Random jumps; cache lines partially wasted	2–10×
Prefetching	Hardware prefetcher handles it	Prefetcher cannot predict	5–10×
Memory-level parallelism	$P \approx 10$–20 outstanding misses	$P = 1$ (dependent loads)	10–20×
TLB	Sequential pages; excellent coverage	Scattered pages; frequent walks	1–5×
Vectorisation (SIMD)	4–8 elements per instruction	1 element per iteration	4–8×
GPU	perfect, GPU were born for that	not even mention pointer chasing	–

These factors are not all independent — they interact in complex ways, and in practice you do not simply multiply them. But the combined effect, for a large, scattered, memory-bound traversal, routinely reaches 50–200× slower than the equivalent array-based computation. That is not a marginal inefficiency. It is the difference between a code that runs in 10 seconds and one that runs in 30 minutes.

---

10. When are linked lists acceptable?

After all this, one might ask: is there ever a legitimate reason to use a linked list?

In HPC, almost never. The only scenario where a linked-list-like structure is defensible is when the dominant cost is *not* traversal but insertion and deletion at arbitrary positions, and the number of elements is small enough that the per-traversal cost is negligible. This can happen in some **graph algorithms** (to be honest: in Advanced HPC, for the sake of simplicity, I'm using a *kind of* linked list to implement thread-parallel graph traversal), in task scheduling queues, and in certain tree structures. But even in those cases, there are usually better alternatives: pool allocators to control memory layout, intrusive lists to eliminate per-node allocation, or array-based structures with an index-linked approach that preserves contiguity.

The general principle, which this lecture illustrates (thanks for the interesting discussion in the classroom), is one of the recurring themes of this course: **on modern hardware, the data layout is the algorithm.** You cannot reason about the performance of an algorithm without understanding how the data is laid out in memory and how the hardware will access it, and viceversa *(see: the comments about the "if forest" in the slides)*. A linked list and an array may both offer $O(N)$ traversal in the algorithmic sense, but the constant hidden in that $O(N)$ differs by two orders of magnitude. In HPC, constants matter.

There is a deeper lesson here, too, which connects to the hot-and-cold field discussion from the previous lecture. The core idea — that you should keep the data you access together *physically together in memory* — applies at every scale: within a struct (hot/cold field separation), within a data structure (arrays vs. linked lists), and across data structures (struct-of-arrays vs. array-of-structs; *see example codes in memory/ex_4_AoS*vs_SoA/). The linked list is simply the most extreme violation of this principle: it places no constraint whatsoever on where the data lives, and delegates that decision entirely to the allocator.

---

11. What to use instead

For most HPC applications, the alternatives are straightforward:

Plain arrays (or `std::vector` in C++). If you need a sequence of elements and you traverse them linearly, use an array. Full stop. If you need to grow the array dynamically, use `realloc()` or let `std::vector` handle it; the occasional reallocation cost is negligible compared to the traversal savings.

Pool allocators for linked structures. If you genuinely need a pointer-linked structure (a tree, a graph, a task queue), allocate all nodes from a contiguous pool. This recovers spatial locality and eliminates per-node `malloc()` overhead. Many HPC libraries provide pool allocators; it is also straightforward to write one.

Index-linked arrays. Instead of pointers, use integer indices into a contiguous array. The array provides contiguity; the indices provide the linking. This is a common pattern in finite-element codes and particle simulations.

Struct-of-arrays (SoA). If each "node" has multiple fields but you only access a few at a time (the hot/cold pattern), split the fields into separate arrays. This ensures that the hot fields are contiguous and cache-friendly, regardless of how large the cold data is. We have discussed this in the previous lecture.